

Data Representation in NumPy and TensorFlow

The Core Inconsistency

- **Historical Context:** The Python data science ecosystem evolved in stages. **NumPy** was created first as the standard for numerical operations. **TensorFlow** was created much later and, while it interoperates with NumPy, it has its own internal conventions for efficiency.
 - **Key Takeaway:** You must be aware of the different data structures (especially 1D vs. 2D arrays) to ensure your code works correctly.
-

Matrices (2D Arrays)

- A **matrix** is a 2D array of numbers.
- The dimension of a matrix is always specified as **Rows x Columns**.
- **Example (2x3 Matrix):**
 - This matrix has 2 rows and 3 columns.

```
1 # [[1, 2, 3],  
2 #  [4, 5, 6]]  
3 x = np.array([[1, 2, 3], [4, 5, 6]])
```

The Key Difference: 1D vs. 2D Arrays (Vectors)

This is the most important convention to understand for this course.

- **Course 1 (NumPy Standard): 1D Array**
 - We represented a feature vector with a *single* set of square brackets.
 - `x = np.array([200, 17])`
 - This creates a **1D array**. It is just a list and technically has **no rows or columns**. Its shape is simply `(2, )`.
- **Course 2 (TensorFlow Convention): 2D Array**
 - TensorFlow is designed to process data in "batches" (matrices). To represent a *single* training example, we must use a **2D array** (a matrix).
 - We use *double* square brackets to create a **row vector**.
 - `x = np.array([[200, 17]])`
 - This creates a **1x2 matrix** (1 row, 2 columns). This is the standard way to feed a single instance into a TensorFlow model.

Code	Type	Dimensions	Shape
<code>np.array([200, 17])</code>	1D Array (1D Vector)	None	(2,)
<code>np.array([[200, 17]])</code>	2D Array (Row Vector)	1 row, 2 cols	(1, 2)
<code>np.array([[200], [17]])</code>	2D Array (Column Vector)	2 rows, 1 col	(2, 1)

The TensorFlow tensor

- When you pass a NumPy array into a TensorFlow layer, TensorFlow converts it into its own internal data type called a **tf.tensor**.
- A **tensor** is just TensorFlow's optimized way of representing a matrix (or vector, or higher-dimensional data).
- For the purposes of this course, you can think of a **tensor** as being equivalent to a **matrix**.

Data Shapes During Inference

When you pass data through your network, the shape of the tensor will change based on the layers.

1. **Input x:** You pass in a **1x2 matrix** (a tensor with shape (1, 2)).
 - `x = np.array([[200, 17]])`
2. **Layer 1 (3 units):** `a1 = Layer1(x)`
 - The layer takes the 1x2 input and computes 3 activations.
 - The output `a1` will be a **1x3 matrix** (a tensor with shape (1, 3)).
3. **Layer 2 (1 unit):** `a2 = Layer2(a1)`
 - The layer takes the 1x3 input and computes 1 activation.
 - The output `a2` will be a **1x1 matrix** (a tensor with shape (1, 1)).

Converting Data

You can easily convert between TensorFlow Tensors and NumPy arrays.

- **Tensor** → **NumPy:** Use the `.numpy()` method.
 - `a1_numpy = a1.numpy()`